

Exercise sheet 3 — Solutions

Decisions – input, output, and programs that react

Published after the in-class discussion. Read these only once you have tried the sheet.

Part A — Talking to the user

A1 *Predict the f-string*

	prints	why
1	4 squared is 16	anything Python can compute goes inside the braces
2	3.14	two decimal places
3	3	zero decimal places — see below
4	4	width 5, right-aligned: four spaces then the 7
5	Ada	width 5, left-aligned: Ada then two spaces
6	Ada	width 7, centred: two spaces either side
7	2.0	/ always gives a float, even for 4/2
8	1,234,567	the comma is a thousands separator

Number 3 prints 3, not 3.14159 and not 4. `.0f` means “zero decimal places”, so it rounds — and 3.14159 rounds to 3.

Worth noticing. Formatting rounds for display only. The value in the variable does not change: after `print(f"{pi:.0f}")` the variable `pi` is still 3.14159. If you want a rounded *value*, that is `round()`, which is a different thing.

A2 *They forgot int()*

	what happens	why
1	<code>TypeError</code>	cannot add an int to a str
2	no error	<code>"2" + "3"</code> joins to <code>"23"</code> — the “sum” prints as 23
3	<code>TypeError</code>	cannot multiply a str by a float
4	<code>TypeError</code>	cannot compare a str with an int

The fix is the same in all four: wrap the input in `int()` — or `float()` in number 3, since a price has decimals.

```
age = int(input("Age: "))
a = int(input("First number: "))
```

```
price = float(input("Price: "))
n = int(input("How many? "))
```

Number 2 is the dangerous one. The other three stop with an error that names the line. Number 2 runs, prints something that looks like an answer, and is wrong.

Worth noticing. Notice what saved you in 1, 3 and 4: Python refused to guess. A language that quietly converted "2" to 2 would have made numbers 1, 3 and 4 behave like number 2 — and you would have found out much later. Strictness is a feature.

A3 Rewrite as f-strings

```
1. print(f"Total: {total} euros")
2. print(f"Name: {name}, age: {age}")
3. print(f"{mass} / {height} ** 2 = {mass / height ** 2:.2f}")
```

Number 2 no longer needs `str()`: the braces convert for you. Number 3 shows the point of the exercise — the computation goes *inside* the braces, and the formatting goes after the colon.

A4 Ask and answer

```
# ask for the name - this stays text
# ask for the year of birth - this has to become a number
# work out the age
# print the greeting

name = input("Your name: ")
year = int(input("Year of birth: "))

age = 2026 - year

print(f"Hello {name}, you turn {age} this year.")
```

Only the year needed `int()`, because it is the only one we do arithmetic with. The name is used as text, and text is what `input()` already gives us — converting it would be pointless, and `int("Ada")` would raise a `ValueError`.

Worth noticing. “Convert it because it is a number” is the wrong rule. The rule is: convert it because you are about to calculate with it. A postcode or a phone number is made of digits and should stay text — you never add two phone numbers together, and `int()` would eat a leading zero.

A5 A table that stays lined up

```
print(f"{'Coffee':<10}{3:>3} x {1.50:>6.2f} = {3 * 1.50:>7.2f}")
print(f"{'Cornetto':<10}{2:>3} x {1.15:>6.2f} = {2 * 1.15:>7.2f}")
print(f"{'Spremuta':<10}{1:>3} x {3.00:>6.2f} = {1 * 3.00:>7.2f}")
print(f"{'Total':<10}{'':>3} {'':>6} {3*1.50 + 2*1.15 + 3.00:>7.2f}")
```

Most of you will — quite reasonably — put the values in variables first. That is better, and it is what the next lecture makes unavoidable.

Renaming Cornetto to Brioche integrale (17 characters) pushes past the width of 10. The width is a *minimum*: the column grows and the rest of that line shifts right. Nothing is truncated and nothing

crashes — but the table stops being square, which is the honest answer to “did your columns survive”. Widening to `:<18` fixes it.

A6 *sep and end*

```
a-b-c
123
```

Two lines, not three. The second print ends with `end=""` instead of a newline, so `print(3)` continues on the same line and supplies the line break itself.

Part B — Making decisions

B1 *True or False*

1	True	both parts hold
2	True	the first part is False, but or needs only one
3	False	<code>x == 5</code> is True, and not flips it
4	True	a chained comparison: <code>0 < 5 and 5 < 10</code>
5	False	<code>y</code> is 10, so the second part fails and needs both
6	False	"Python" and "python" differ in the first character

For number 6: `s.lower() == "python"` is the usual answer. We meet `.lower()` properly in lecture 5; anyone who found it deserves the credit.

B2 *Which branch runs?*

	prints	why
1	fine	30 is not <i>greater than</i> 30, and not less than 10
2	big and medium	two separate if statements, both True
3	nothing	if <code>n</code> : on the number 0 is False
4	pass and excellent	again two separate if statements

Number 2 prints two lines because the second if is a new question, asked regardless of how the first one went. Changing the second if to `elif` gives one line — and changes the meaning of the program, which is the point.

Number 4 is not necessarily wrong. A mark of 30 really is both a pass and a distinction, so printing both may be exactly what you want. Separate ifs and `elif` answer different questions; neither is the correct one in general.

Worth noticing. Number 3 is a preview. Python treats `0` as False and any non-zero number as True, and the same for empty versus non-empty text. It is convenient and it is a trap — for now, write the comparison you mean (`if n != 0:`) rather than relying on it.

B3 *Wrong, but no error*

1 — always True. Python reads it as `(mark == 28) or (30)`. The second part is the bare number 30, which counts as True, so the whole condition is always True and *every* mark prints distinction.

```
if mark == 28 or mark == 30:
```

If you print the broken condition you see something odd: `print(mark == 28 or 30)` shows 30, not True. or does not return a yes/no answer — it returns the first operand that counts as true, and here that is the bare 30. In an if that is all it takes.

2 — the second branch is unreachable. An amount over 100 is also over 50, so the first branch catches it and the 20% band never runs. Test with 150: it gives 10%. Put the most specific test first:

```
if amount > 100:
    discount = 0.20
elif amount > 50:
    discount = 0.10
else:
    discount = 0
```

3 — two messages at once. With `temp = 35` it prints hot *and* fine, because the third condition (`temp >= 10`) is also true. Three independent questions were asked where one three-way choice was meant:

```
if temp > 30:
    print("hot")
elif temp < 10:
    print("cold")
else:
    print("fine")
```

Worth noticing. Three bugs, three different causes: an operator that does not mean what English means, branches in the wrong order, and separate questions where one choice was intended. All three run. All three are silent. This is what “it works” is worth as evidence.

B4 Ticket price

```
# under 6 is free
# 6 to 17 pays 5
# 18 to 64 pays 12
# 65 and over pays 8
# print the price

age = int(input("Age: "))

if age < 6:
    price = 0
elif age < 18:
    price = 5
elif age < 65:
    price = 12
else:
    price = 8

print(f"Price: {price} euros")
```

age	5	6	17	18	64	65	100
price	0	5	5	12	12	8	8

The two that catch people are **6** and **18** — the first age of each new band. Writing `age <= 6` or `age < 6` is a one-character difference that moves a six-year-old from free to paying.

Note how few comparisons this needs. Once `age < 6` has failed, we already know the age is at least 6, so the next test only has to check the upper end. If you wrote `elif age >= 6 and age < 18:` you are not wrong, but doing work the `elif` already did.

B5 Two conditions

```
user = input("Username: ")
password = input("Password: ")

if user == "ada" and password == "lovelace":
    print("Welcome")
else:
    print("Access denied")
```

Nested:

```
if user == "ada":
    if password == "lovelace":
        print("Welcome")
    else:
        print("Access denied")
else:
    print("Access denied")
```

The `and` version is better: one condition, one fallback, and `Access denied` written once. In the nested version the same line appears twice, so a later change has to be made in two places — and one day it will be made in only one.

Worth noticing. The nested version also leaks information. It can tell you *which* of the two was wrong, and a real login screen deliberately does not, because that turns one unknown into two separate guessing games.

B6 The discount, properly

```
# over 100 gets 20 percent
# over 50 gets 10 percent
# otherwise nothing
# work out what there is to pay

amount = float(input("Amount: "))

if amount > 100:
    discount = 0.20
elif amount > 50:
    discount = 0.10
else:
    discount = 0

to_pay = amount * (1 - discount)
```

```
print(f"Discount: {discount * 100:.0f}%")
print(f"To pay: {to_pay:.2f}")
```

amount	discount	to pay
40.00	0%	40.00
50.00	0%	50.00
50.01	10%	45.01
100.00	10%	90.00
150.00	20%	120.00

50.01 is in the sheet to show that the boundary is real and one cent wide. 100.00 still gets only 10%, because the rule says *more than* 100 — exactly the trap from sheet 1, A7, now in a program that will happily charge it.

B7 Air quality

```
co2 = int(input("CO2 in ppm: "))

if co2 < 800:
    print("fine")
elif co2 <= 1400:
    print("getting stuffy")
else:
    print("open a window")
```

a) With the version above, 800 gives getting stuffy and 1400 gives getting stuffy.

b) Yes — the rules say “800 to 1400”, which reads as including both ends. Anyone who wrote `elif co2 < 1400:` has 1400 falling through to open a window, which contradicts the stated rule.

c) Either fix is acceptable, *as long as it is stated*. The interesting answer is that the rules themselves are sloppy: “below 800” and “800 to 1400” and “above 1400” only tile the number line if you read the middle band as inclusive at both ends. Real specifications are like this constantly, and the job is to notice and ask.

Worth noticing. This is the whole point of the boundary exercises. The program cannot be more precise than the specification it came from. When the specification is vague, you are not choosing an implementation — you are making a decision on someone else’s behalf, and you should say so out loud.

If you finish early

Adding the weekend question doubles the branches:

```
if age < 6:
    price = 0
elif age < 18:
    price = 7 if weekend else 5
...
```

or, worse and more common, a nested `if weekend:` inside every band.

The fix is not a better if. It is to stop writing the same decision four times — which is what functions (lecture 8) are for, and what a table of prices (lecture 6 and 7) makes possible. Keep the program and bring it back then.